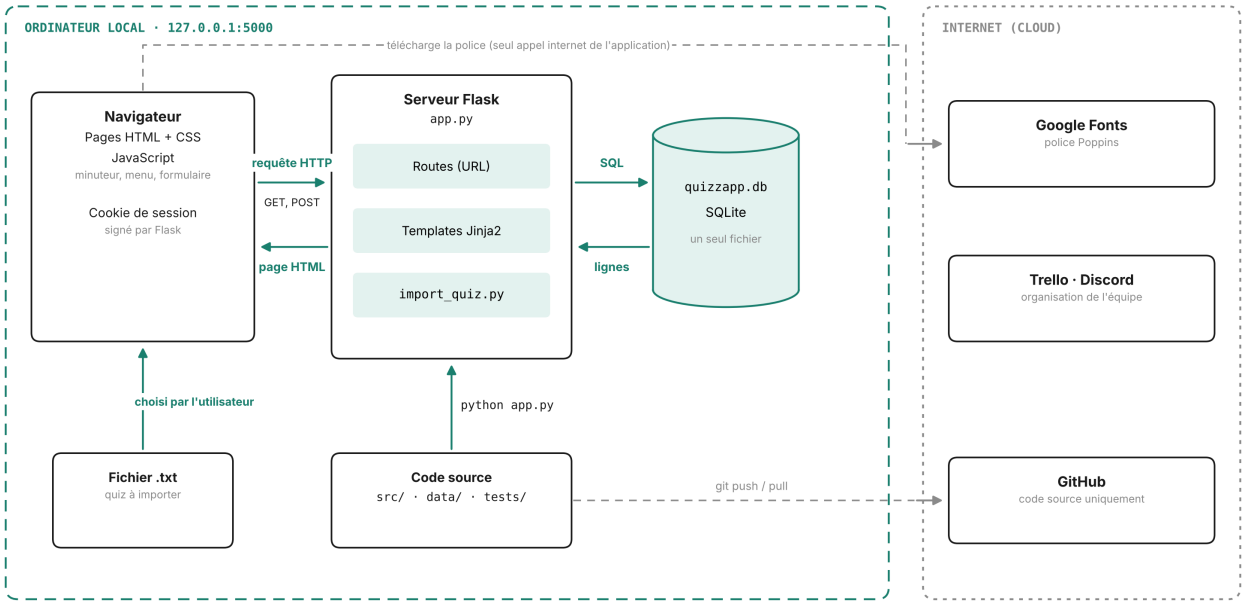
Architecture de QuizApp

PROJ531 – Groupe 4 – Application Quiz Python · 23/10/2026

Une application web client-serveur écrite en Python avec Flask. Elle tourne entièrement sur l'ordinateur où on la lance, et ses données sont stockées dans un fichier SQLite local.

Architecture	Données	Cloud
Client-serveur, un seul programme Flask (monolithe)	En local, dans le fichier quizzapp.db (SQLite)	Seulement GitHub, pour partager le code dans l'équipe

1. Vue d'ensemble



Le navigateur, le serveur Flask et la base SQLite sont sur le même ordinateur. Seuls le code (sur GitHub) et la police d'écriture passent par internet : aucune donnée d'utilisateur ne quitte la machine.

2. Parcours d'une requête : importer un quiz

1. L'utilisateur choisit un fichier .txt sur la page « Importer ». Le navigateur l'envoie au serveur (requête POST), avec le cookie de session.
2. Flask lit le cookie pour vérifier que l'utilisateur est connecté.
3. Le module `import_quiz.py` lit le texte ligne par ligne et vérifie le format. En cas d'erreur, la page est renvoyée avec la liste des problèmes et leur numéro de ligne.
4. Si tout est correct, le quiz est enregistré dans quizzapp.db par une requête SQL paramétrée.
5. Flask redirige vers l'accueil. Jinja2 génère la page HTML avec le nouveau quiz, puis le navigateur l'affiche.

3. Où sont stockées les données

Donnée	Emplacement	Détail
Comptes utilisateurs	Local · SQLite	Table utilisateurs : identifiant, mot de passe (role et score seront retirés, voir le MCD)
Quiz créés ou importés	Local · SQLite	Table provisoire quizzes, qui sera remplacée par les tables du MCD (quiz, questions, réponses)
Quiz d'exemple	Local · mémoire	Liste FAUX_QUIZ dans app.py, pour tester l'interface. Elle sera supprimée.
Session (qui est connecté)	Navigateur · cookie	Cookie signé par Flask avec une clé secrète : impossible à modifier sans que le serveur le détecte
Fichier importé	Local · mémoire	Lu puis oublié : seul le quiz extrait est enregistré dans la base
Code source	Cloud · GitHub	Partagé entre les 4 membres. La base quizzapp.db n'y est pas : chacun a la sienne.

4. Pourquoi en local et pas dans le cloud

Ce que ça nous apporte	Les limites, connues
• **Rien à installer** : SQLite est inclus dans Python (module sqlite3) ; la base est un simple fichier créé au premier lancement. • **Chacun travaille seul** : chaque membre lance python app.py sur son ordinateur, avec sa propre base de test. • **Gratuit et hors ligne** : pas d'hébergeur, pas d'abonnement, pas de compte cloud à gérer. • **Données protégées** : les comptes et les scores ne sont envoyés à aucun service extérieur. • **Adapté au projet** : la démonstration finale se fait sur un seul ordinateur.	• L'application n'est accessible que depuis l'ordinateur qui la lance. • Les bases des membres ne sont pas partagées : un quiz créé chez l'un n'apparaît pas chez l'autre. • SQLite n'est pas fait pour beaucoup d'utilisateurs qui écrivent en même temps. • Les mots de passe sont encore stockés en clair ; leur hachage est prévu au sprint 4.

Et pour passer au cloud ? Il suffirait d'héberger l'application Flask chez un hébergeur Python et de remplacer SQLite par un serveur de base de données comme PostgreSQL. Les routes, les pages et le module d'import resteraient les mêmes : seule la connexion à la base changerait, avec quelques ajustements SQL.

5. Modèle de données

Aujourd'hui (provisoire) : utilisateurs (login, password, score, role) et quizzes (id, titre, description, duree, questions en JSON, visibilite, createur, code_partage). Toutes les questions d'un quiz sont rangées en texte JSON dans une seule colonne.

Cible (MCD du 23/10) : quiz, questions et réponses dans des tables séparées, reliées par leurs identifiants, plus une table des tentatives pour l'historique. Identifiant souligné, clé étrangère précédée de #.

- **UTILISATEUR** (login, mot_de_passe)
- **QUIZ** (id_quiz, titre, description, duree, visibilite, code_acces, date_creation, #login_createur)
- **QUESTION** (id_question, enonce, #id_quiz)
- **REPONSE** (id_reponse, texte, est_correcte, #id_question)
- **TENTATIVE** (id_tentative, score, date_tentative, #login, #id_quiz)

6. Technologies utilisées

Rôle	Technologie	Pourquoi
Langage	Python 3	Langage du cours, connu de toute l'équipe.
Serveur web	Flask	Framework léger : routes, sessions et messages en quelques lignes.
Pages	HTML + Jinja2	Gabarit commun (base.html) dont héritent toutes les pages.
Style	CSS, police Poppins	Une seule feuille de style, style.css.
Interactivité	JavaScript, sans framework	Minuteur du quiz, menu mobile, ajout de questions dans le formulaire.
Base de données	SQLite (sqlite3)	Requêtes paramétrées, qui protègent contre les injections SQL.
Import de quiz	Python (import_quiz.py)	Format texte simple : QUIZ:, Q:, +, -.